SYSTEM ANALYSIS & DESIGN REPORT

SynapX GymOS
End-to-End System Analysis

A reverse-engineered analysis of the SynapX GymOS codebase — a multi-tenant gym operations platform — covering its architecture, entity–relationship model, class structure, use cases, application screenshots, and a ready-to-write final report chapter outline.

3 sub-systems | 14 database tables | 6 staff roles

Node · Express · Prisma · PostgreSQL | React · Vite

Python · FastAPI · InsightFace

SynapX GymOS — synapx-backend · synapx-biometric-service · synapx-gymos

Generated from a direct read of the project source: prisma/schema.prisma, all route / controller / service files, and app.py

01 System overview

SynapX GymOS is a multi-tenant gym operations platform. A single deployment serves many gyms (**tenants**), each with one or more physical locations (**branches**). Staff run the business — members, billing, classes, equipment — from an admin dashboard, while members check themselves in at an unattended kiosk using QR, face recognition, fingerprint, or manual lookup.

The codebase is split into three independently deployable services:

BACKEND API

`SynapX_Backend_API` — Node.js + Express REST API. Owns all business data via Prisma ORM over PostgreSQL. Issues JWTs, enforces role-based access, talks to Stripe and the biometric service.

BIOMETRIC SERVICE

`SynapX_Biometric_Service` — Python + FastAPI microservice. Runs face embedding/matching (InsightFace) and bridges an optional ZKTeco fingerprint reader. Never touches the main database directly.

FRONTEND

`SynapX_GymOS_Frontend` — React 18 + Vite admin dashboard (Members, Classes, Payments, Staff, Attendance, Reports) plus the public Kiosk check-in screen.

ACTORS

ACTOR	NATURE	WHAT THEY DO
Super Admin	internal	Full control, incl. enrolling/removing member biometrics.
Admin	internal	Runs day-to-day operations: members, billing, classes, staff.
Manager	internal	Membership plans, equipment, branch operations.
Trainer	internal	Own class schedule, booking attendance.
Receptionist	internal	Front-desk: check-in support, bookings, member lookup.
Read-Only	internal	View-only access, e.g. auditors.
Member	external	Self-service kiosk check-in; no login account.
Biometric Device	external	Webcam, ZKTeco reader, or the device's own platform sensor (Windows Hello / Touch ID via WebAuthn).
Stripe	external	Sends payment-status webhooks back to the API.

02 System architecture

Three services, three storage roles: PostgreSQL for the business record, an in-memory/disk face index inside the biometric service, and Stripe as the external payment authority.

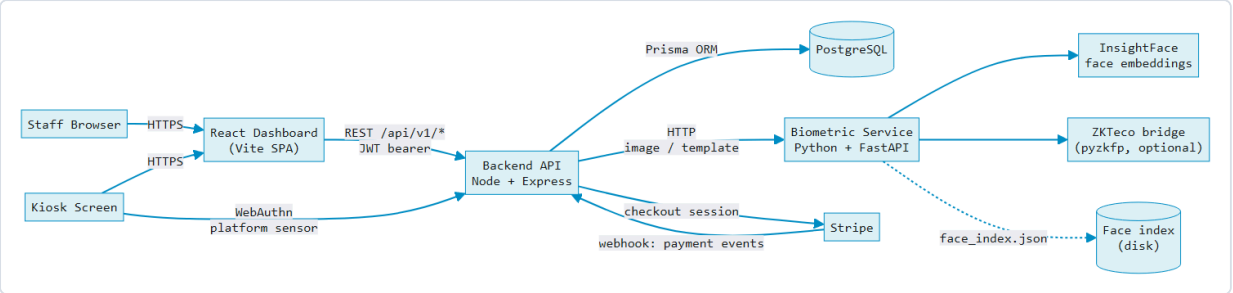


Fig. 1 — Service topology. The backend never stores or matches biometric data itself; it forwards captures to the Python microservice and persists only the bytes returned.

NOTABLE DESIGN DECISIONS

- **Two independent fingerprint paths.** A legacy path forwards raw templates to a ZKTeco desk reader bridged by `pyzkfp`; the primary path uses **WebAuthn** so a laptop's own sensor (Windows Hello, Touch ID) can authenticate a member — the server only ever sees a public key, never a template.
- **Attendance is denormalized on purpose.** `AttendanceLog` has no foreign keys to `Member` / `Branch` — it stores copies of the name/code at check-in time, so kiosk logging never fails even if a lookup or a later data change would otherwise block it.
- **Multi-tenancy is scoped through the data model**, not separate databases: every `Branch` hangs off a `Tenant`, and most staff/report queries are scoped to `req.user.branchId`.
- **Role gating is middleware-based** (`protect`, then `adminOnly` / `adminUp` / `managerUp` / `trainerUp` / `receptionUp`), stacking cleanly on top of route handlers rather than being re-checked inside each controller.

03 Entity-relationship diagram

Derived directly from `prisma/schema.prisma` — 14 tables. Fields are trimmed to the ones that carry the relationships and business meaning.

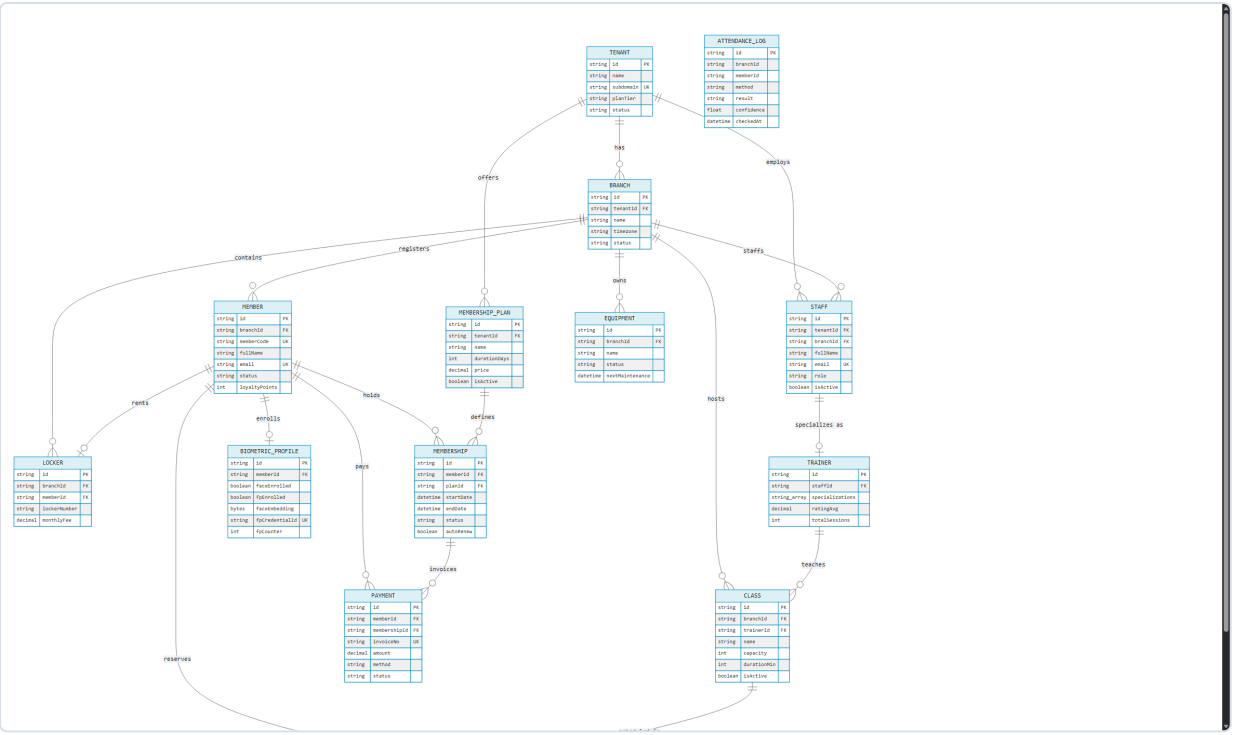


Fig. 2 — 13 relational tables plus the intentionally standalone `ATTENDANCE_LOG` (no enforced FK — see architecture notes above).

ENUMERATED TYPES

ENUM	VALUES
`PlanTier`	STARTER · PRO · ELITE · ENTERPRISE
`MemberStatus`	ACTIVE · FROZEN · SUSPENDED · EXPIRED · CANCELLED
`PaymentStatus`	PENDING · PAID · OVERDUE · REFUNDED · CANCELLED
`PaymentMethod`	CASH · CARD · ONLINE · BANK_TRANSFER
`StaffRole`	SUPER_ADMIN · ADMIN · MANAGER · TRAINER · RECEPTIONIST · READ_ONLY
`BookingStatus`	CONFIRMED · WAITLIST · CANCELLED · ATTENDED · NO_SHOW
`EquipmentStatus`	ACTIVE · MAINTENANCE · BROKEN · RETIRED

04 Class diagrams

Two views: the persistent domain model (mirrors the Prisma schema as UML classes, with the operations each entity's routes actually expose), and the service-layer collaboration behind biometric check-in.

4.1 — DOMAIN MODEL

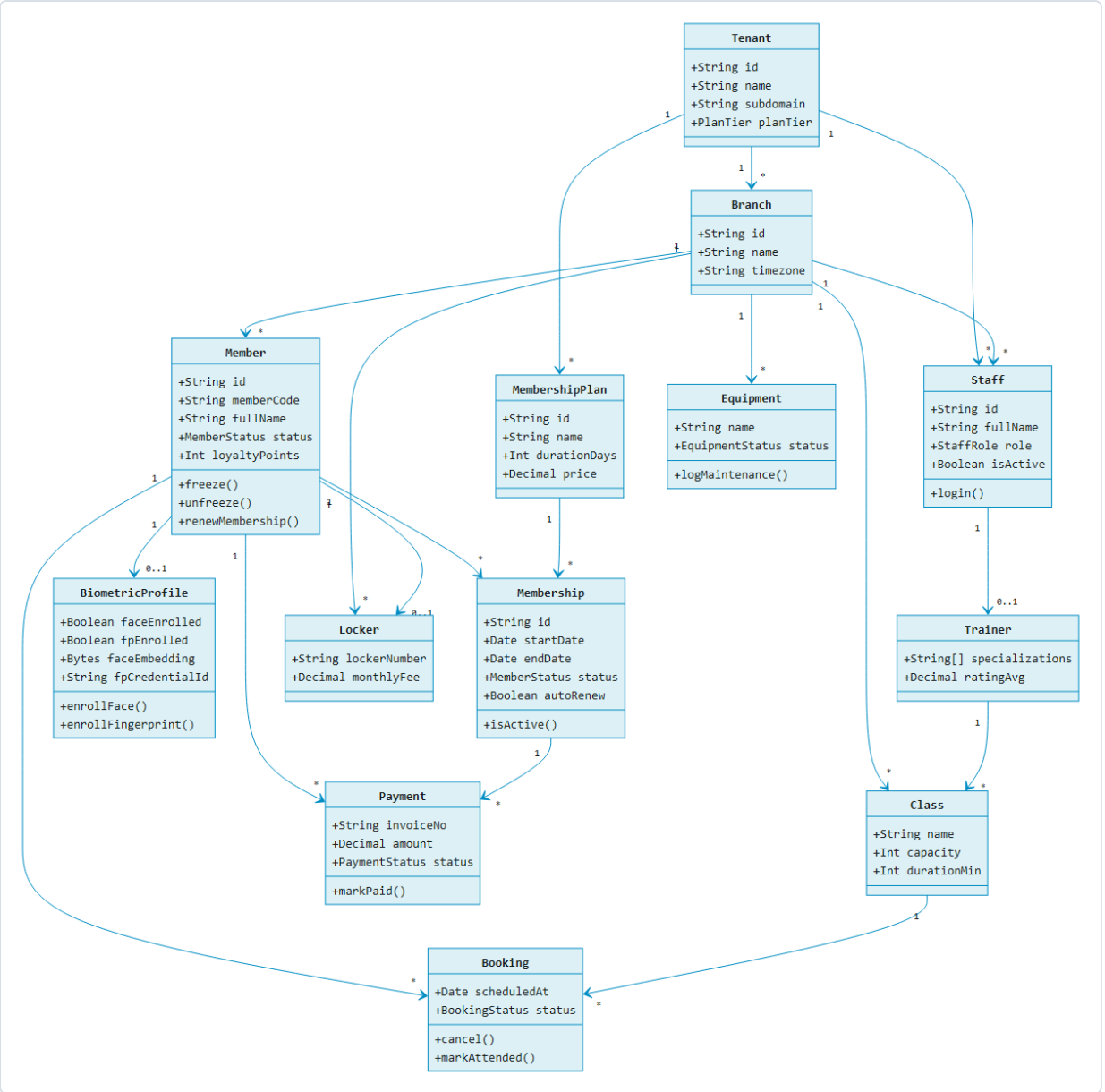


Fig. 3 — Operations shown on each class are the real mutations its controller exposes (e.g. `PATCH /members/:id/freeze` → `Member.freeze()`), not invented behavior.

4.2 — BIOMETRIC CHECK-IN: SERVICE COLLABORATION

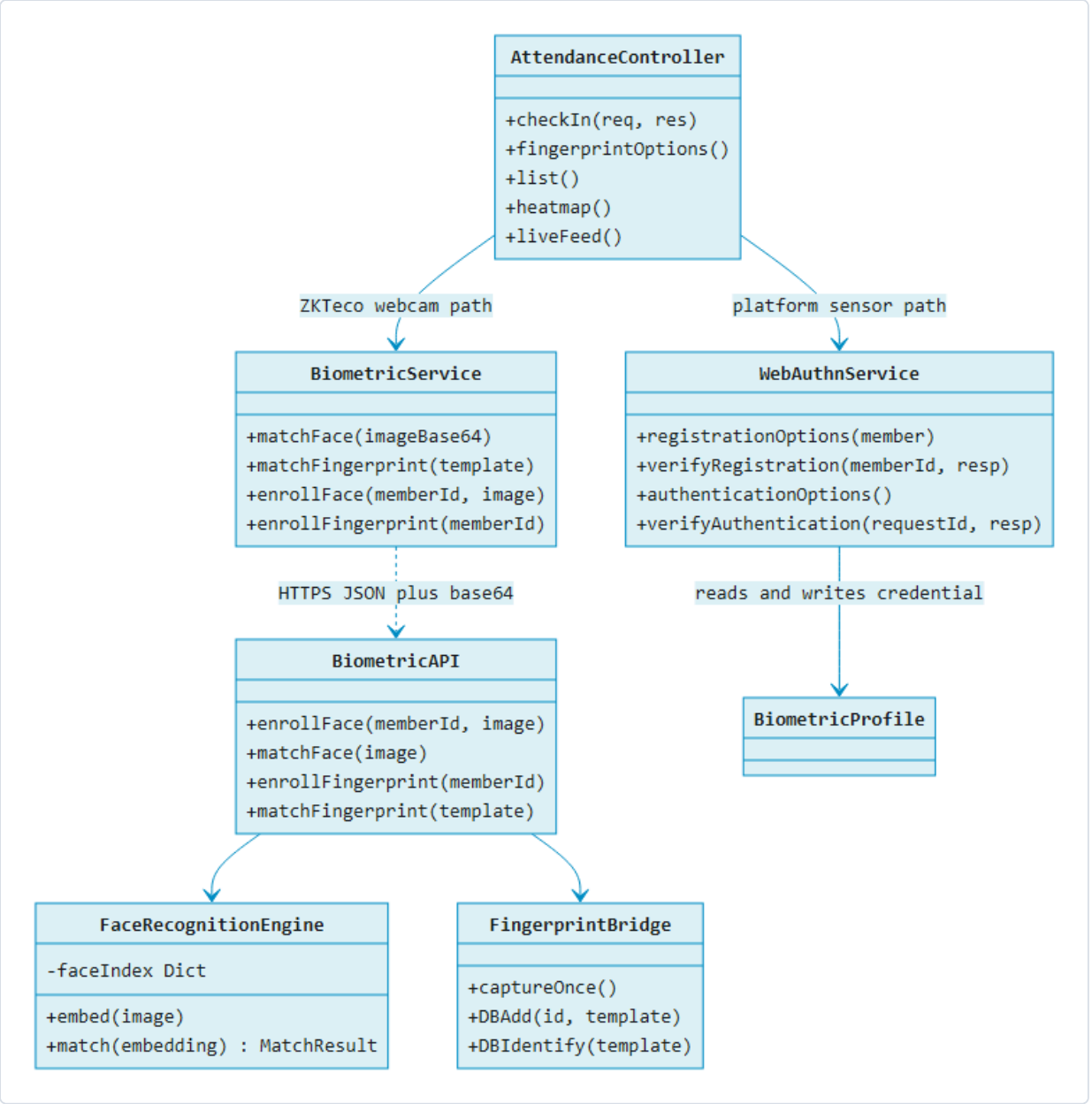
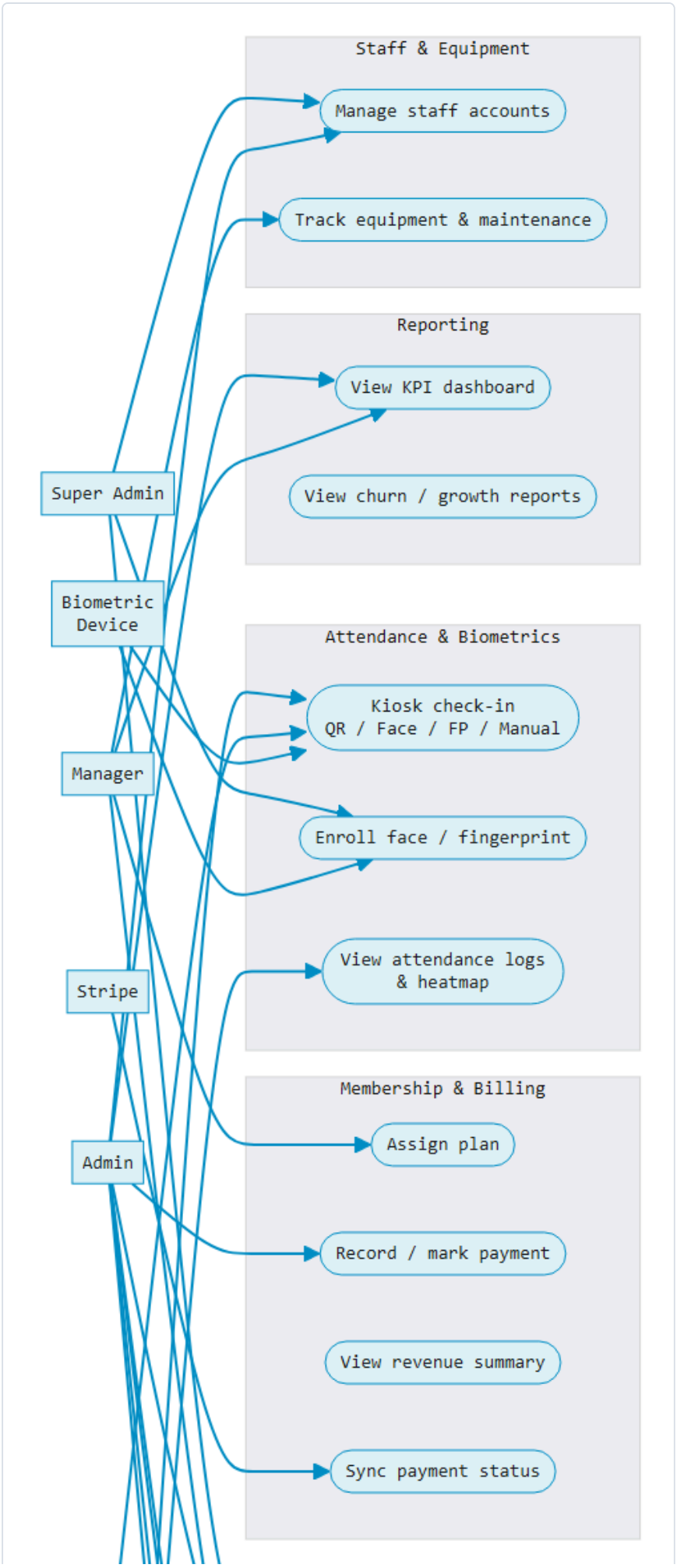


Fig. 4 — Two parallel fingerprint strategies converge on one `checkIn()` pipeline; the WebAuthn path never leaves member devices with a template, only a public key.

05 Use case diagram

Actors and use cases grouped by module, plus the complete role-accurate endpoint mapping below.


Staff & Equipment
Manage staff accounts
Track equipment & maintenance
Reporting
View KPI dashboard
View churn / growth reports
Super Admin
Biometric Device
Attendance & Biometrics
Kiosk check-in
QR / Face / FP / Manual
Manager
Enroll face / fingerprint
View attendance logs & heatmap
Stripe
Membership & Billing
Assign plan
Admin
Record / mark payment
View revenue summary
Sync payment status

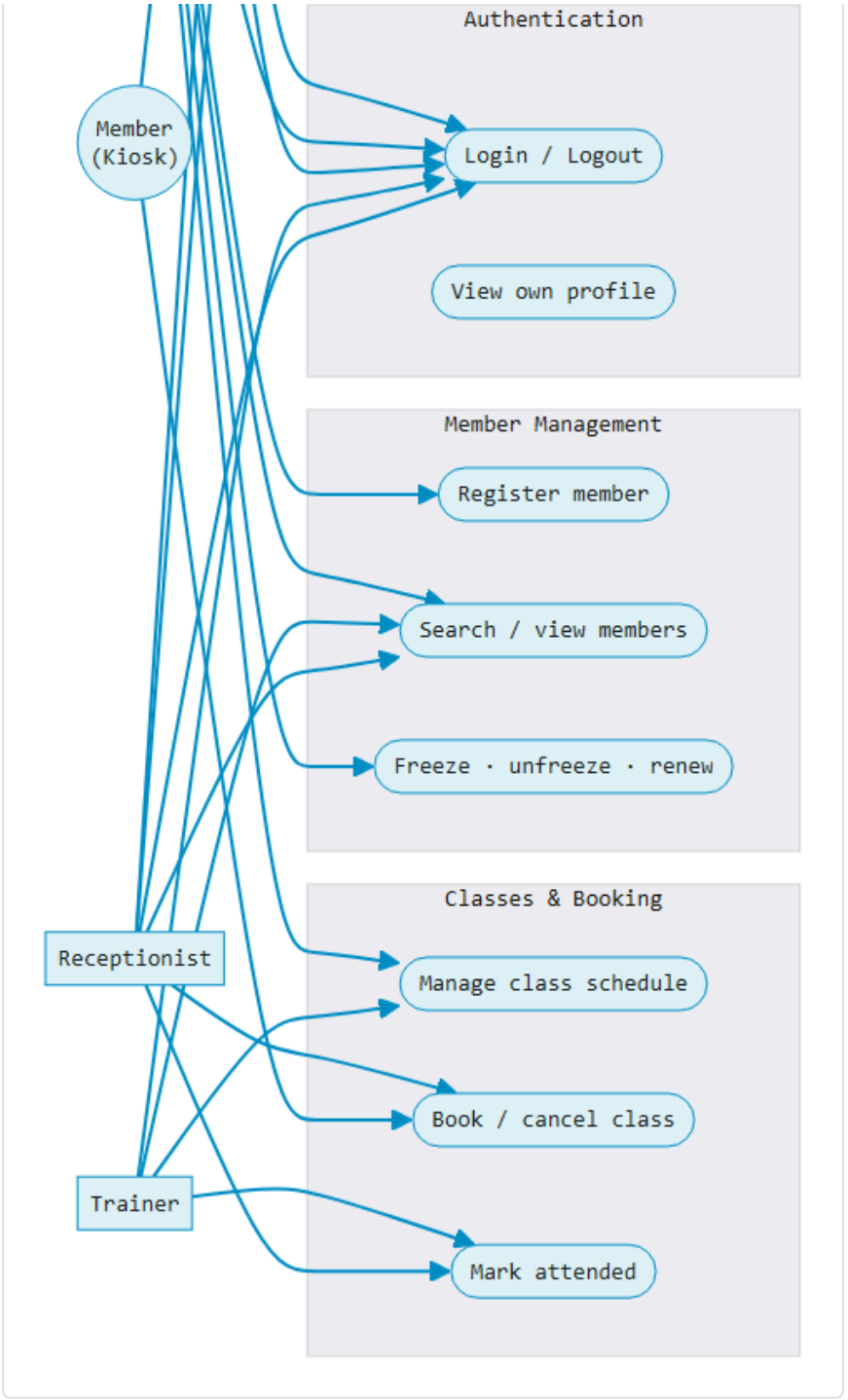


Fig. 5 — Actor-to-module use case overview. Precise per-endpoint role gates in the table below.

FULL USE-CASE → ROLE MAPPING

MODULE	USE CASE	ACTOR(S)	ENDPOINT
Auth	Log in / refresh / log out	`any staff`	`POST /auth/login`
Auth	View own profile	`any staff`	`GET /auth/me`
Members	Search / list / view member	`any staff`	`GET /members`

MODULE	USE CASE	ACTOR(S)	ENDPOINT
Members	Register, update, delete, freeze, renew	Admin+	POST·PUT·DELETE /members
Memberships	List plans / view member's plan	any staff	GET /memberships
Memberships	Define plan, assign, renew	Manager+	POST /memberships
Payments	View invoices / revenue summary	any staff	GET /payments
Payments	Record payment, mark paid	Admin+	POST /payments
Payments	Stripe payment sync	Stripe (external)	POST /webhooks/stripe
Attendance	Kiosk check-in (QR/Face/FP/Manual)	Member (unauthenticated)	POST /attendance/checkin
Attendance	View logs, today stats, heatmap, live feed	staff	GET /attendance/*
Biometrics	Enroll / verify / remove profile	Super Admin	POST·DELETE /biometric/:id
Classes	View schedule	any staff	GET /classes
Classes	Create, update, delete class	Admin+	POST·PUT·DELETE /classes
Bookings	Book, cancel, mark attended	Member / Receptionist / Trainer	POST /bookings
Staff	List / view staff	any staff	GET /staff
Staff	Create, update, delete staff	Admin+	POST·PUT·DELETE /staff
Equipment	View equipment	any staff	GET /equipment
Equipment	Add, update, log maintenance	Manager+	POST·PATCH /equipment
Reports	KPI dashboard, revenue/growth/plan charts	any staff	GET /dashboard/*
Reports	Member, revenue, churn, monthly reports	any staff	GET /reports/*

06 Application screenshots

Captured from the running system: backend API on PostgreSQL, frontend dashboard on Vite, seeded demo data for FitNation Gym.

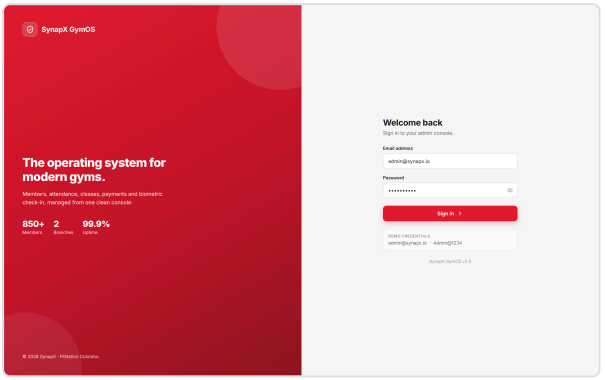


Fig. 6 — Staff sign-in, JWT issued on submit.

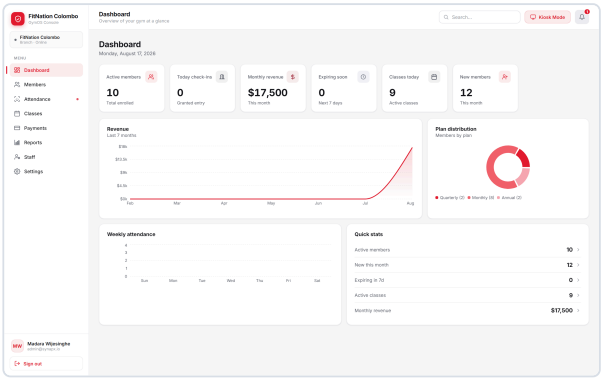


Fig. 7 — KPI dashboard: active members, revenue, plan distribution, weekly attendance.

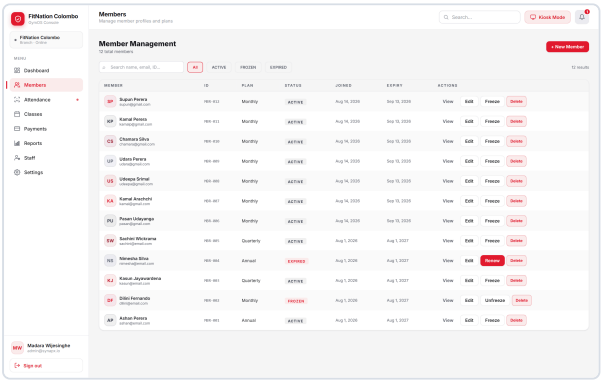


Fig. 8 — Member management: search, status filters, freeze/renew actions.

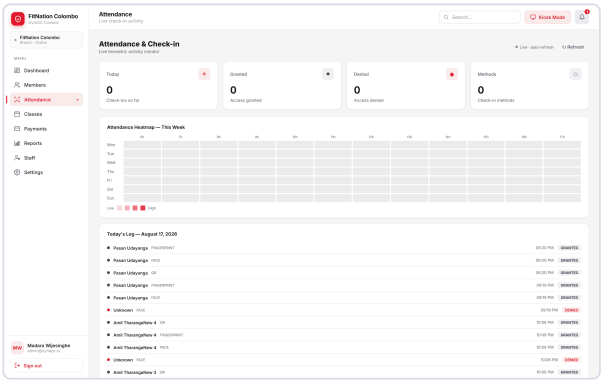


Fig. 9 — Live attendance monitor with weekly heatmap and today's check-in log.

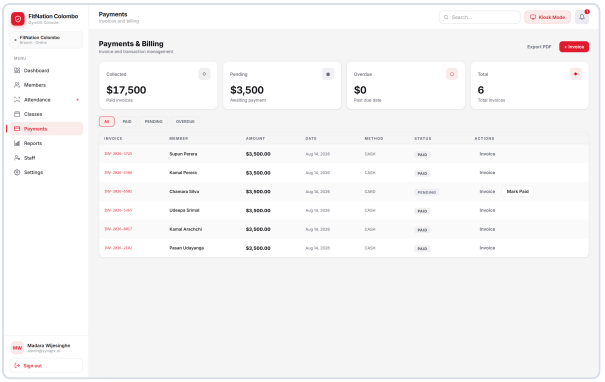


Fig. 10 — Payments & billing: invoice list, status filters, mark-paid action.

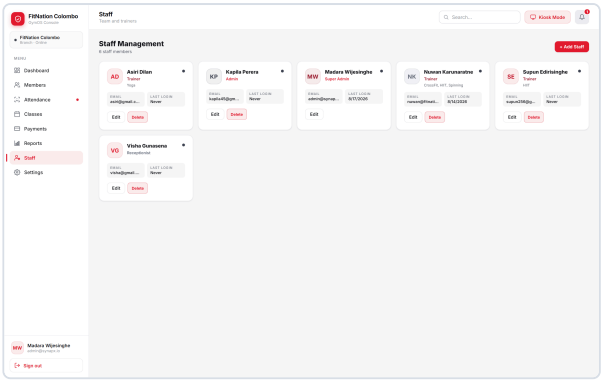


Fig. 11 — Staff management: role badges (Trainer / Admin / Super Admin / Receptionist).

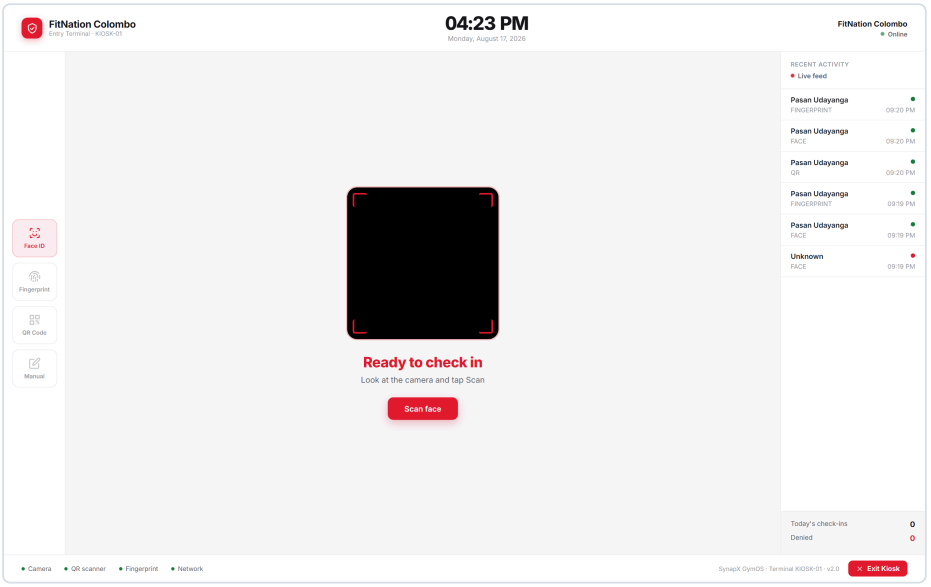


Fig. 12 — Unattended kiosk: Face ID / Fingerprint / QR / Manual check-in, with a live activity feed.

07 Final report — chapter outline

A working skeleton for the written report, with content bullets grounded in what the codebase actually does.

01 Introduction

- **Background:** gyms managing members, billing, and attendance across paper/spreadsheets or disconnected tools.
- **Problem statement:** no single system ties membership status, billing, and physical access (check-in) together in real time.
- **Objectives:** multi-tenant member/billing management; biometric + QR attendance without extra hardware cost; role-based staff operations; live KPI reporting.
- **Scope:** single-organisation gym chains (tenant → branches); web dashboard + unattended kiosk; explicitly out of scope: native mobile app, member self-service portal.

02 Literature / comparable systems

- Compare against commercial gym-management SaaS (e.g. Mindbody, Glofox) — what they offer for billing/attendance/scheduling.
- Biometric access control precedent — face recognition (ArcFace/InsightFace embeddings) and WebAuthn/FIDO2 as a template-free alternative to dedicated fingerprint hardware.

03 Requirements

- **Functional:** the 20 use cases in §05, grouped by module (Auth, Members, Billing, Attendance, Classes, Staff, Equipment, Reporting).
- **Non-functional:** multi-tenant data isolation; RBAC across 6 roles; JWT-based stateless auth; kiosk check-in must degrade gracefully when the biometric service is unreachable.

04 System design

- Architecture diagram & rationale for splitting the biometric workload into its own Python service (Fig. 1).
- ER diagram and schema rationale — why `AttendanceLog` is denormalized (Fig. 2).
- Class diagrams — domain model and the biometric service collaboration (Figs. 3–4).
- Use case diagram and role/permission matrix (Fig. 5).
- *Recommended addition:* a sequence diagram for the kiosk check-in flow, since it is the system's most stateful interaction.

05 Implementation

- **Backend:** Express 4, Prisma 5 over PostgreSQL, JWT, bcrypt password hashing, Stripe SDK, PDFKit for invoices, Winston logging.
- **Biometric service:** FastAPI, InsightFace (`buffalo_l` , cosine similarity threshold 0.45), optional `pyzkfp` ZKTeco bridge, disk-persisted face index.
- **Frontend:** React 18 + Vite, React Router, Recharts, `@simplewebauthn/browser` , `jsqr` for QR scanning.
- Note the two-generation architecture visible in the repo: the backend README still documents a MongoDB/Redis-era design, while the current schema moved attendance logging into PostgreSQL.

06 Testing

- No automated test suite was found in any of the three services — flag this and either add one (Jest/Supertest, Pytest) or document manual test cases per use case in §05.
- Suggested manual cases: expired-membership check-in is denied; frozen member check-in is flagged; face match below threshold falls back to manual; role-gated routes reject a lower-privilege token with 403.

07 Results & discussion

- Screenshots of the dashboard, kiosk check-in, and a sample invoice (§06).
- Discuss the WebAuthn-over-ZKTeco decision as a cost/UX trade-off worth highlighting as a contribution.

08 Conclusion & future work

- Objectives met vs. §01; limitations (single-currency assumptions, no mobile app, no automated tests).
- Future work: member self-service portal, automated test coverage, multi-currency billing, class waitlist promotion logic.

Generated from a direct read of SynapX_Backend_API, SynapX_Biometric_Service, and SynapX_GymOS_Frontend — prisma/schema.prisma, all route/controller/service files, and app.py.